Moodify — Team Guide

Nov 12, 2026 · @Aditya

Overview

Moodify turns a sentence about how someone feels, in English, Hindi or Hinglish, into a ranked list of Hindi and English songs. This guide settles every shared decision up front so five people can build in parallel without breaking each other's code.

Read **Shared decisions** before writing any code, then your own section under **Roles**. Where this guide and the README disagree, this guide wins and the README is updated to match.

Team at a glance

Each person owns one part end to end. Below, the two Adityas are written as **Aditya J** and **Aditya KY**.

Member	Roll no.	Role	Owns
Aditya Kumar Yadav	12340100	NLP evaluation, API contracts and QA	Keyword fallback engine, Hinglish benchmark, prompt examples, evaluation harness, request validation, data mappers, app mock data, fallback pool, Profile screen, test plan, README, slide deck, merges
Aditya Jha	12340090	Backend lead and LLM	Auth middleware, emotion detection service, mood and preferences endpoints, demo video
Niraj Chandekar	12341480	Recommendations and data	Scoring, recommendation and feedback endpoints, song catalog and links, deployment, report compilation

Member	Roll no.	Role	Owns
Sanjani Kumari	12341890	Mobile: mood to music flow	API client, Mood AI screen, Recommendations screen, song components
Hirannya Mhaisbadwe	12340950	Mobile: auth and dashboards	Login, sign-up, onboarding, Home, My Music, My Mood, demo build

Shared decisions

These are fixed for the whole project. To change one, agree in the group first; Aditya KY then updates this section and the README.

1. Naming and values

All JSON keys are **camelCase**, in every request and response (`confirmedMood` , `targetMoods` , `preferredLanguages` , `createdAt`). The database keeps snake_case. The backend converts between them in one file, `backend/src/utils/mappers.js` , and nowhere else.

Field	Allowed values	Example
Mood (`confirmedMood` , `targetMoods` , a song's `moods` , the `mood` query)	Happy, Sad, Angry, Stressed, Relaxed, Motivated, Romantic, Nostalgic. Title Case, exactly these 8	`"Relaxed"`
`energy` , `intensity`	low, medium, high. Lowercase	`"low"`
`emotion` , `context`	Free text, lowercase snake_case. Unknown: `neutral` , `general`	`"work_pressure"`
Song `language`	Hindi, English	`"Hindi"`
`language` filter (query)	Hindi, English, Both	`"Both"`
Feedback `type`	like, dislike, skip	`"like"`
Analysis `source`	llm, keyword	`"keyword"`

Field	Allowed values	Example
Recommendation `source`	db, fallback	`"db"`
Dates	ISO 8601 strings in UTC	`"2026-11-13T09:30:00Z"`
IDs	UUID strings	

The requirements doc shows moods in lowercase. We use Title Case because the song catalog and the app's mood constants already do.

2. Response format

Every endpoint returns one of two shapes:

```
{ "data": { } }
{ "error": { "code": "VALIDATION_ERROR", "message": "text must be 1-500 characters" } }
```

HTTP	`code`	When
400	`VALIDATION_ERROR`	A field is missing or has a value not allowed above
401	`UNAUTHORIZED`	Token missing, expired or invalid
404	`NOT_FOUND`	The mood entry or song does not exist, or is not yours
404	`PROFILE_NOT_FOUND`	The user has not finished onboarding; the app opens onboarding
500	`INTERNAL_ERROR`	Anything else. Never send stack traces

A failed LLM call is **not** an error. `/mood/analyze` still returns 200, with `"source": "keyword"`.

3. Endpoints

Base path `/api`. Every endpoint needs `Authorization: Bearer <Supabase access token>`.

Endpoint	Owner	Request	`data` in the response
`POST /mood/analyze`	Aditya J	`{ text }`, 1-500 characters after trimming	`MoodAnalysisResult`
`POST /mood/confirm`	Aditya J	`{ text, confirmedMood, analysis }`; `analysis` = the object `/mood/analyze` returned, sent back unchanged	`{ entry: MoodEntry }`
`GET /mood/history?limit=20`	Aditya J	`limit` 1-100, default 20	`{ entries: MoodEntry[], weeklyCounts }`. Entries newest first. `weeklyCounts` covers the last 7 days and lists all 8 moods, 0 when none
`DELETE /mood/:id`	Aditya J	`:id` = a UUID, or `all`	`{ deleted: number }`
`GET /preferences`	Aditya J	none	`UserProfile`, or 404 `PROFILE_NOT_FOUND`
`PUT /preferences`	Aditya J	`{ name, preferredLanguages, genres }`	`UserProfile`. Creates the profile if it does not exist
`GET /recommendations?mood=Relaxed&language=Both&limit=10`	Niraj	`mood` required. `language` defaults to the profile's languages. `limit` 1-20, default 10	`{ songs: Song[], source }`. Every song has `score` and `reason`
`POST /feedback`	Niraj	`{ songId, type }`	`{ songId, type }`. One row per user and song; a new tap replaces the old one
`GET /feedback/liked`	Niraj	none	`{ songs: Song[] }`, newest like first

Two changes from the README:

- `/mood/confirm` saves the entry and returns only `{ entry }`. The Recommendations screen always loads songs itself with `GET /recommendations`. Songs come from one place only.
- `GET /feedback/liked` is new. My Music needs it.

Type changes in `mobile/src/types/index.ts`: `MoodAnalysisResult` gains `source: 'llm' | 'keyword'`. No other type changes.

4. Backend function signatures

```
// services/keywordFallback.js  (Aditya KY)
keywordFallback(text)  // -> MoodAnalysisResult. Synchronous, under 50 ms,
never throws. source: 'keyword'

// services/emotionService.js  (Aditya J)
analyzeMood(text)  // -> Promise<MoodAnalysisResult>. Never rejects.
                   //    Gemini first; keywordFallback on a 3 s timeout, an
API error or invalid JSON

// services/recommendationService.js  (Niraj)
getRecommendations({ userId, mood, language, limit = 10 })
                   // -> Promise<{ songs, source: 'db' | 'fallback' }>. Never
rejects; fallback pool on a database error

// utils/scoring.js  (Niraj)
scoreSong(song, { mood, languages, likedArtists, likedGenres })  // -> {
score, reason }
```

`getRecommendations` takes one object, not separate arguments, so new options can be added later without breaking callers.

Rules for all of them:

- The LLM and the keyword fallback return the **same shape**. Only `source` tells them apart.
- `targetMoods` holds 1-3 of the 8 moods, strongest first. `targetMoods[0]` is the mood the app shows as detected.
- If no keyword matches, the fallback returns `{ emotion: "neutral", energy: "medium", intensity: "low", context: "general", targetMoods: ["Relaxed"], source: "keyword" }`.
- Nothing throws to the caller. Log failures with `console.warn` and return the fallback.

5. Song links: YouTube

Every song's `url` is a YouTube link of the form `https://www.youtube.com/watch?v=<id>`. YouTube plays the full song free and without an account, on any phone or emulator. Spotify needs its app or a login, which can break a live demo.

- Use the official upload: the label's channel (T-Series, Sony Music India, Zee Music and so on) or the artist's official or VEVO channel. Lyric re-uploads get taken down.
- Open every link once before committing it. All 120 placeholder Spotify links are replaced.
- The app opens a link with `Linking.openURL(url)`. That opens the YouTube app if installed, otherwise the browser.

6. Song IDs

Every song in `songs.seed.json` gets a fixed UUID in place of ids like `h-01`. The loader inserts songs with those ids. `fallback_songs.json` lists the ids of 20 seed songs, and the recommendation service reads their details from `songs.seed.json`. A song then has one id everywhere, and each link is stored only once.

7. Login and profile flow

1. Sign-up and login happen in the app, directly with Supabase Auth (email and password), using the anon key. Sign-up asks only for email and password.
2. The app sends the session's access token as `Authorization: Bearer <token>` on every backend call.
3. The backend checks the token with `supabase.auth.getUser(token)` and sets `req.user = { id, email }`.
4. After login the app calls `GET /preferences`. `PROFILE_NOT_FOUND` opens onboarding; anything else opens Home.
5. Onboarding asks for name, languages and genres and saves them with `PUT /preferences`, which creates the profile.

Mood entries and feedback reference the profile, so nothing can be saved before onboarding is done.

8. Other conventions

- **LLM:** Gemini only (`LLM_PROVIDER=gemini`), in JSON output mode with a response schema. The model in `.env.example` (`gemini-1.5-flash`) is outdated. Aditya J picks the current Flash model in Google AI Studio and updates `.env.example` .
- **Timeouts:** the backend gives the LLM 3 s. The app gives each request 10 s, then shows an error with a retry button.
- **Scoring weights:** mood 0.5, language 0.3, feedback 0.2, as in the README.
- **Home mood chips:** tapping a chip opens Recommendations for that mood directly. It does not call `/mood/analyze` and saves no mood entry.
- **Backend code:** CommonJS (`require`), as `server.js` already uses. File names in camelCase, such as `emotionService.js` .
- **Mobile code:** TypeScript. Components in `src/components` named `PascalCase.tsx` ; hooks in `src/hooks` named `useSomething.ts` .
- **Secrets:** `.env` files are never committed. The service role key stays on the backend; the app only ever gets the anon key.
- **Wording:** a mood is self-reported and AI-estimated ("Sounds like you're feeling Relaxed"), never a diagnosis. No clinical words such as "depression" in the app.

Roles

Each task is tagged with its phase (P1 to P6, see **Phases and merge order**). Finish a phase's tasks before its merge round.

Aditya Jha: Backend lead and LLM

What to do

- **P1:** Get the Gemini API key. Check that `npm run dev` starts and `/health` answers.
- **P2:** `config/supabase.js` , `middlewares/auth.js` , `middlewares/errorHandler.js` , and the route wiring in `server.js` and `routes/index.js` .
- **P3:** `services/emotionService.js` (`analyzeMood`), built on Aditya KY's few-shot examples. `POST /mood/analyze` , `POST /mood/confirm` , `GET` and `PUT /preferences` .
- **P4:** `GET /mood/history` with weekly counts and `DELETE /mood/:id` . Fix bugs that mobile reports.
- **P5:** Tune the prompt using Aditya KY's error analysis until accuracy is at least 80%. If his latency numbers put `/mood/analyze` over 3 s, fix it.

- **P6:** Record the demo video. Write your report section and slides.

Be careful

- The service role key skips row-level security. Filter **every** query by `req.user.id`, never by a user id from the request body.
- Put Aditya KY's `validate` middleware on every route, and use `mappers.js` for every database read and write. Don't add separate checks or conversions.
- Check the LLM's JSON against the allowed values before returning it. Anything else goes to `keywordFallback`.
- The prompt must say the input may be English, Hindi (Devanagari) or Romanised Hinglish, with examples of each.
- `DELETE /mood/all` deletes only the caller's entries.

Niraj Chandekar: Recommendations and data

What to do

- **P1:** Create the Supabase project, run the migration, and share the keys in the group chat (not in git).
- **P2:** Write `backend/scripts/seedSongs.js` and load the 120 songs. They already carry fixed UUIDs from P1.
- **P3:** `utils/scoring.js`, `services/recommendationService.js`, `GET /recommendations`, `POST /feedback`, `GET /feedback/liked`. On a database error, serve the songs listed in `fallback_songs.json`. Scoring:
  - **Mood match:** only songs tagged with the mood are candidates; 1.0 when it is the song's first mood tag, 0.7 when it is a later one.
  - **Language match:** 1 when the song's language is in the user's preferred languages, else 0.
  - **Feedback boost:** 1 when the user liked another song by the same artist, 0.5 for the same genre, else 0.
- **P4:** Replace all 120 links with YouTube links. Write the reason strings.
- **P5:** Deploy the backend to Render and share the URL. Build the Emotional Journey playlist (stretch goal) only if everything else works.
- **P6:** Collect everyone's report sections and compile the report. Write your own section and slides.

Be careful

- Remove disliked songs **after** scoring, so nothing can add them back.
- The `language` filter is strict: `Hindi` returns only Hindi songs. `Both` returns either.
- Feedback is an upsert on (user, song). The table has a unique constraint, so a plain insert fails on the second tap.
- Reasons stay under 60 characters and state facts, such as "Fits your Relaxed mood · you like Arijit Singh".
- Render's free tier sleeps when idle, so the first request after a while is slow. Open the URL a few minutes before the demo.

Sanjani Kumari: Mobile, mood to music flow

What to do

- **P1:** Install the app's packages and open it in Expo Go.
- **P2:** `src/services/supabase.ts` and `src/services/api.ts`. The API client adds the token, returns `data`, and throws an `ApiError` carrying `code` when it gets `error`. When `EXPO_PUBLIC_USE_MOCKS=true` it returns Aditya KY's mock data from `src/services/mocks.ts` instead of calling the backend. Also build `src/components/MoodChip.tsx`.
- **P3:** `src/hooks/useMood.ts` and the Mood AI screen (`app/(tabs)/mood.tsx`): text box, loading state, detected mood, and a modal to confirm it or pick another. Built on mock data.
- **P4:** Switch to the real API. Build `app/recommendations.tsx` (takes a `mood` route param), `useRecommendations`, `SongCard`, `FeedbackButtons`, and opening links.
- **P5:** Language filter (Hindi, English, Both). Loading, error and empty states. A short note when fallback songs are shown.
- **P6:** Test someone else's screens. Write your report section and slides.

Be careful

- The app never calls Gemini and never sees the service role key.
- Send `/mood/confirm` the exact `analysis` object that `/mood/analyze` returned.
- Disable the submit button while a request runs, so one tap cannot save two entries.
- A disliked song leaves the list at once and comes back if the request fails.
- A physical phone cannot reach `localhost`. Set `EXPO_PUBLIC_API_BASE_URL` to your laptop's local network IP, or to the deployed URL.

- Show the mood as an estimate ("Sounds like you're feeling Relaxed"), never as a diagnosis.

Hirannya Mhaisbadwe: Mobile, auth and dashboards

What to do

- **P1:** Install the app's packages and open it in Expo Go.
- **P2:** Login and sign-up screen layouts.
- **P3:** `src/context/AuthContext.tsx` and the auth gate in `app/_layout.tsx`: no session opens login, no profile opens onboarding, otherwise the tabs. Onboarding asks for name, languages and genres and saves them with `PUT /preferences`. Build the pickers as reusable components (`LanguagePicker`, `GenrePicker`), because Profile reuses them.
- **P4:** Home: greeting, mood chips (using `MoodChip`) and the last 3 entries. My Music: liked songs from `GET /feedback/liked`.
- **P5:** My Mood: weekly counts, the entry list, delete one entry, delete all.
- **P6:** The Expo or EAS build for the demo. Test someone else's screens. Write your report section and slides.

Be careful

- Store the Supabase session with AsyncStorage so users stay logged in after closing the app. Tell the group when you add the package.
- "Delete all" asks for confirmation first.
- Weekly counts come from the server. Do not recount them in the app.
- Use `MoodChip` and the colours and emoji in `src/constants/moods.ts` everywhere, so moods look the same on every screen.

Aditya Kumar Yadav: NLP evaluation, API contracts and QA

What to do

- **P1:** Publish the API contract: README section 6, and `source` added to `MoodAnalysisResult` in `mobile/src/types/index.ts`. Assign a stable UUID to each of the 120 catalog songs in `songs.seed.json`.
- **P2:** Build the contract layer that the backend and the app code against:

    - `services/keywordFallback.js` v1: a rule-based multilingual classifier over the 8-mood taxonomy, with English and Romanised Hinglish lexicons, returning the shared shape.
    - `middlewares/validate.js`: request validation for every endpoint, returning `VALIDATION_ERROR`.
    - `utils/mappers.js`: the single snake_case to camelCase translation layer.
    - `mobile/src/services/mocks.ts`: mock responses matching the contract, so the app can be built before the backend is ready.
    - `data/fallback_songs.json`: the offline fallback pool of 20 songs, balanced across all 8 moods and both languages.
- **P3:** Extend the lexicon to Devanagari. Build the three-language benchmark `tests/hinglish_eval.json` (about 150 labelled sentences). Curate the few-shot examples for Aditya J's prompt in `services/promptExamples.js`. Draft the end-to-end test plan in `docs/test-plan.md`.
- **P4:** `tests/emotion_detection.test.js`: an evaluation harness for both the LLM and the fallback. It reports accuracy overall and per language, plus p50 and p95 latency. Send the misclassified sentences to Aditya J.
- **P5:** Re-run after prompt tuning; confusion matrix and error analysis. Profile screen (`app/(tabs)/profile.tsx`): edit preferences with Hirannya's pickers, and log out.
- **P6:** Run the team test session against the test plan. Final README (tracker ticked, evaluation results). Assemble the slide deck from everyone's slides. Report chapter "NLP methodology and evaluation". The last merge and the v1.0 tag.
- **Every phase:** as integration owner, merge PRs in the agreed order and check that `main` still runs.

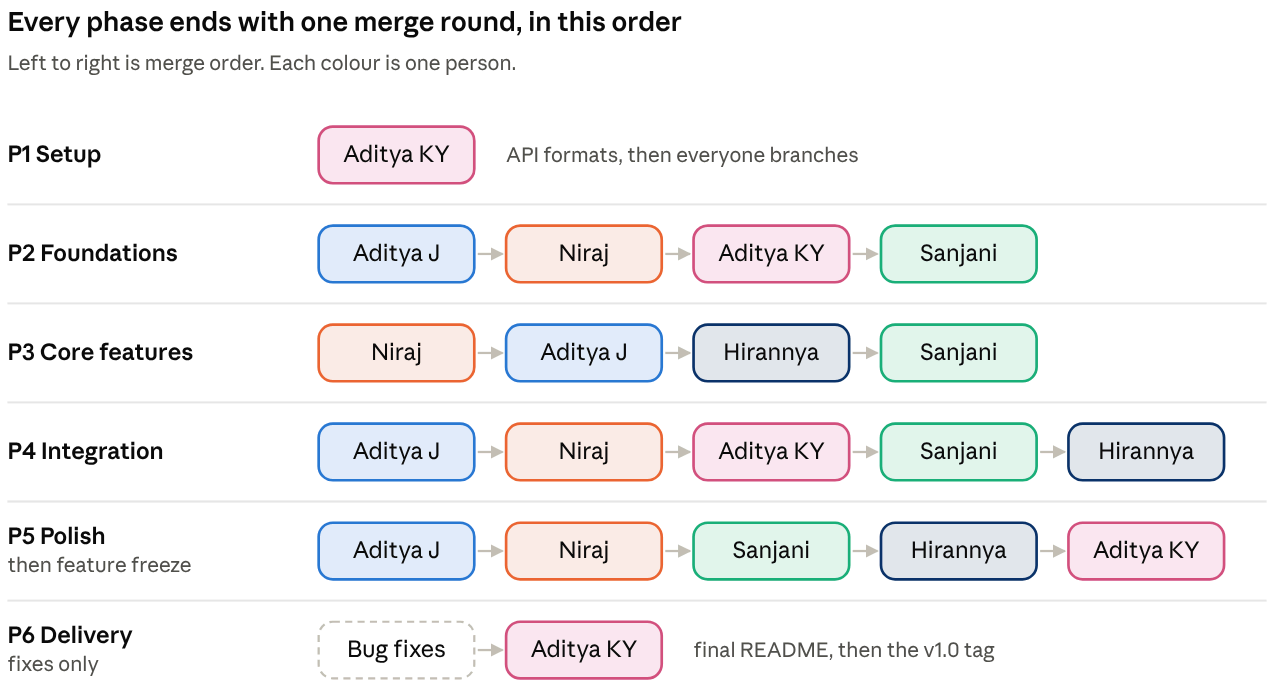
Be careful

- Merge everything in the P2 contract layer even if rough. The backend and the app both build on it.
- Mocks must match the endpoint table field for field, or the app breaks when it switches to the real API.
- Hinglish spelling varies (`bahut`, `bohot`, `bhot`). Lowercase the input, strip punctuation, and list the common spellings.
- Handle simple negation: "not happy" and "khush nahi" must not count as Happy.

- Write benchmark sentences the way people actually text: typos, mixed scripts. If an LLM drafts them, rewrite them by hand, because LLM-written text is easier for an LLM and inflates accuracy.
- Never take few-shot examples from the benchmark. A prompt that has seen the test sentences inflates accuracy.
- Freeze the benchmark after the first run so before and after numbers compare. Never change a label to raise accuracy.
- A live run makes about 150 Gemini calls. Add a short delay between calls to stay under the free-tier rate limit.

Phases and merge order

The work runs in 6 phases. Each phase ends with one merge round in a fixed order, and the next phase starts from the merged `main`.



merge order · 6 phases

Backend pieces merge before the screens that call them, so later PRs in a round can rely on earlier ones.

After each merge round, check the line below before moving on:

Phase	Goal	Check before moving on
P1 Setup	Decisions written down, everyone set up	The backend starts, the app opens in Expo Go, everyone has the keys
P2 Foundations	Shared plumbing	`/health` answers; a request without a token gets 401; the 120 songs are in the database
P3 Core features	Main backend and first screens	With Postman: text returns a mood, and a mood returns songs. In the app: sign-up leads to onboarding
P4 Integration	App on the real API	On a phone: type a feeling, confirm it, get songs, open one on YouTube
P5 Polish	Everything built; features frozen after this	Accuracy at least 80%; backend deployed; every screen has loading, error and empty states
P6 Delivery	Ship	Every test-plan item passes on real phones, including a bad Gemini key and a stopped backend; v1.0 tagged; report, slides and demo video done

Git rules

- One branch per task, named `feat/<area>` (`feat/emotion-service`, `feat/mood-screen`). Nobody pushes to `main` directly.
- Commit messages start with the type and area: `feat(backend): ...`, `fix(mobile): ...`.
- **Before a merge round:** rebase on `main`, resolve your own conflicts, and check that your part runs. A PR that isn't ready waits for the next round.
- Aditya KY merges, in the order shown above. If a merge breaks `main`, it is reverted at once and the author fixes it on their branch.
- Never commit `.env` files or keys.
- **Shared files have one owner.** To change one, ask its owner.